CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN

Bài 01: Thuật toán và Lý thuyết độ phức tạp thuật toán

Nguyễn Thị Tâm
nguyenthitam.hus@gmail.com

Ngày 28 tháng 1 năm 2026

Nội dung

1. Bài toán và thuật toán
2. Độ phức tạp thuật toán
3. Các ký pháp
4. Xác định độ phức tạp tính toán
5. P và NP

Nội dung

1. Bài toán và thuật toán
2. Độ phức tạp thuật toán
3. Các ký pháp
4. Xác định độ phức tạp tính toán
5. P và NP

Bài toán

Bài toán trong tin học là một vấn đề cần giải quyết thông qua việc sử dụng các phương pháp và kỹ thuật tin học, chẳng hạn như lập trình, thuật toán, cấu trúc dữ liệu, và các lý thuyết tính toán.

- Bài toán tìm ước chung lớn nhất của hai số nguyên dương
  - Cho: Hai số nguyên dương a, b
  - Tìm: UCLN u (là số lớn nhất mà cả a, b đều chia hết)
- Bài toán giải phương trình bậc hai một ẩn
  - Cho: Ba số a, b, c
  - Tìm: Phương trình có nghiệm hay không? Nếu có thì có bao nhiêu nghiệm, và giá trị của các nghiệm đó là bao nhiêu?
- Bài toán sắp xếp một dãy số theo thứ tự tăng dần
  - Cho: Dãy số $a = (a_1, a_2, ..., a_n)$
  - Tìm: Dãy số a với các phần tử được sắp xếp theo thứ tự tăng dần
- ...

Bài toán

- Bài toán được cấu tạo bởi hai thành phần cơ bản:
  - Đầu vào (*input*): cung cấp các dữ liệu đã có
  - Đầu ra (*output*): những yếu tố cần tìm
- Cho bài toán: cho *input* và *output*
- Giải bài toán: từ *input*, dùng các thao tác thích hợp để tìm được *output*

Khái niệm thuật toán (*Algorithm*)

Định nghĩa

Thuật toán (thuật giải/giải thuật) để giải một bài toán là một dãy hữu hạn các thao tác được sắp xếp theo một trình tự nhất định sao cho sau khi thực hiện dãy các thao tác đó, từ đầu vào của bài toán ta nhận được đầu ra.

Ví dụ:

- Thuật toán tương ứng với một công thức nấu ăn
- Thuật toán giải phương trình bậc hai $ax^2 + bx + c = 0$

Các đặc trưng của thuật toán

- Tính *xác định thứ tự*: các bước của thuật toán phải xác định rõ bước nào thực hiện trước, bước nào thực hiện sau
  - Mặc dù vậy, không nhất thiết yêu cầu phải thực hiện các dãy bước tuần tự bước 1, bước 2,. . . (ví dụ, các thuật toán song song)

Các đặc trưng của thuật toán

- Tính *xác định thứ tự*: các bước của thuật toán phải xác định rõ bước nào thực hiện trước, bước nào thực hiện sau
  - Mặc dù vậy, không nhất thiết yêu cầu phải thực hiện các dãy bước tuần tự bước 1, bước 2,. . . (ví dụ, các thuật toán song song)
- Tính *hiệu quả*: các bước của thuật toán phải *thực hiện được*
  - Ví dụ, thuật toán không thể chứa bước nào đó có dạng "*Tạo một dãy gồm tất cả các số nguyên dương*", vì dãy số nguyên dương là vô hạn, bước này không thực hiện được.

Các đặc trưng của thuật toán

- Tính *xác định thứ tự*: các bước của thuật toán phải xác định rõ bước nào thực hiện trước, bước nào thực hiện sau
  - Mặc dù vậy, không nhất thiết yêu cầu phải thực hiện các dãy bước tuần tự bước 1, bước 2,. . . (ví dụ, các thuật toán song song)
- Tính *hiệu quả*: các bước của thuật toán phải *thực hiện được*
  - Ví dụ, thuật toán không thể chứa bước nào đó có dạng "*Tạo một dãy gồm tất cả các số nguyên dương*", vì dãy số nguyên dương là vô hạn, bước này không thực hiện được.
- Tính *không nhập nhằng*
  - Thông tin ở mỗi trạng thái trong quá trình thực hiện thuật toán phải rõ ràng và đầy đủ để xác định chính xác các hành động cần thực hiện ở mỗi bước của thuật toán..

Các đặc trưng của thuật toán

- Tính *xác định thứ tự*: các bước của thuật toán phải xác định rõ bước nào thực hiện trước, bước nào thực hiện sau
  - Mặc dù vậy, không nhất thiết yêu cầu phải thực hiện các dãy bước tuần tự bước 1, bước 2,. . . (ví dụ, các thuật toán song song)
- Tính *hiệu quả*: các bước của thuật toán phải *thực hiện được*
  - Ví dụ, thuật toán không thể chứa bước nào đó có dạng "*Tạo một dãy gồm tất cả các số nguyên dương*", vì dãy số nguyên dương là vô hạn, bước này không thực hiện được.
- Tính *không nhập nhằng*
  - Thông tin ở mỗi trạng thái trong quá trình thực hiện thuật toán phải rõ ràng và đầy đủ để xác định chính xác các hành động cần thực hiện ở mỗi bước của thuật toán..
- Tính *hữu hạn*: thuật toán phải xác định một quá trình có tính *kết thúc*, nghĩa là thuật toán phải dừng sau một số hữu hạn bước.

Thuật toán và biểu diễn của thuật toán

Cần nhấn mạnh sự khác nhau giữa một thuật toán và biểu diễn của nó. Tương tự như giữa một cốt truyện và một cuốn sách.

- Cốt truyện : trừu tượng, mang tính khái niệm;
- Cuốn sách : là biểu diễn vật lí của cốt truyện. Khi truyện được dịch sang thứ tiếng khác, hoặc xuất bản dưới dạng khác thì chỉ phần biểu diễn là thay đổi, cốt truyện vẫn không đổi.

Thuật toán và biểu diễn của thuật toán

Cần nhấn mạnh sự khác nhau giữa một thuật toán và biểu diễn của nó. Tương tự như giữa một cốt truyện và một cuốn sách.

- Cốt truyện : trừu tượng, mang tính khái niệm;
- Cuốn sách : là biểu diễn vật lí của cốt truyện. Khi truyện được dịch sang thứ tiếng khác, hoặc xuất bản dưới dạng khác thì chỉ phần biểu diễn là thay đổi, cốt truyện vẫn không đổi.

Thuật toán có thể được biểu diễn dưới nhiều dạng khác nhau. Ví dụ, thuật toán chuyển từ nhiệt độ C sang nhiệt độ F :

- Biểu diễn bằng công thức đại số : $\boxed{F = \frac{9}{5}C + 32}$;
- Biểu diễn bằng chỉ dẫn : *Nhân nhiệt độ C với* $\frac{9}{5}$ *và cộng với 32.*

Các cách biểu diễn thuật toán thường dùng

- Ngôn ngữ tự nhiên
- Sơ đồ khối
- Mã giả
- Ngôn ngữ lập trình

Dùng ngôn ngữ tự nhiên

Ví dụ:

- Thuật toán Euclid tìm ước chung lớn nhất của hai số nguyên dương x và y được diễn đạt như sau:

  Chừng nào cả x và y khác 0, chia giá trị lớn hơn cho giá trị nhỏ hơn và thay x, y tương ứng bởi số chia và phần dư. Giá trị cuối cùng của x là ước chung lớn nhất.

Dùng ngôn ngữ tự nhiên

Ví dụ: Bài toán tìm giá trị lớn nhất của một dãy số gồm N phần tử

1. Định nghĩa bài toán
   - Input: Số nguyên dương n và dãy gồm n số nguyên $a_1,\ldots,a_n$
   - Output: Giá trị lớn nhất của dãy số max
2. Thuật toán
   - Bước 1: $max := a_1; i := 2;$
   - Bước 2: Kiểm tra nếu $i > n$ thì trả lại giá trị max rồi kết thúc
   - Bước 3:
     - 3.1 Nếu $a_i > max$ thì $max := a_i$
     - 3.2 Tăng $i := i + 1$ rồi quay lại Bước 2

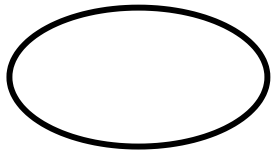
Dùng ngôn ngữ tự nhiên

Ví dụ: Kiểm tra số nguyên N có phải là số nguyên tố hay không?

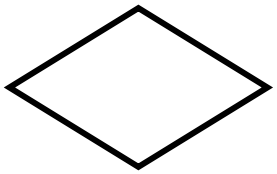
1. Định nghĩa bài toán
   - Input: Số nguyên n
   - Output: Giá trị Đúng (true) nếu n là số nguyên tố, giá trị Sai (false) nếu n không phải là số nguyên tố
2. Thuật toán
   - Bước 2: if n $<$ 2 then return false
   - Bước 3: for $i := 2 \rightarrow n - 1$
     - if $n\%i == 0$ then return false
   - Bước 4: return true

Dùng sơ đồ khối

- Vẽ sơ đồ biểu diễn thuật toán theo hệ thống kí hiệu trực quan được quy ước trước.



Bắt đầu, kết thúc



Điều kiện

Thao tác tính toán

Khối nhập xuất dữ liệu

Luồng xử lý

Dùng mã giả

- Mã giả (*pseudo-code*) là một hệ thống các kí hiệu, thuật ngữ dùng để biểu diễn thuật toán.
- Là sự trừu tượng hóa của ngôn ngữ lập trình, không phụ thuộc vào ngôn ngữ lập trình cụ thể nào.
- Được xây dựng dựa trên các cấu trúc ngữ nghĩa nhằm diễn tả quá trình phát triển của thuật toán.

Thuật toán và biểu diễn của thuật toán

Cần phân biệt 2 khái niệm *chương trình* và *quá trình*

- Mỗi chương trình (*program*) là biểu diễn của một thuật toán dùng cho máy tính.
- Mỗi quá trình (*process*) là hành động chạy một chương trình, cũng là chạy một thuật toán.

Câu hỏi/Bài tập

1. Giải thích vì sao các bước mô tả dưới đây không tạo thành một thuật toán:

   Bước 1. Lấy một đồng xu ra khỏi ví và đặt nó lên bàn.

   Bước 2. Trở lại bước 1.

Phát hiện thuật toán

Phát triển một thuật toán gồm 2 giai đoạn:

- Phát hiện thuật toán
- Biểu diễn thuật toán dưới dạng chương trình

Phương pháp phát hiện thuật toán? Vấn đề chính trong quá trình phát triển phần mềm.

Phát hiện thuật toán

Phát triển một thuật toán gồm 2 giai đoạn:

- Phát hiện thuật toán
- Biểu diễn thuật toán dưới dạng chương trình

Phương pháp phát hiện thuật toán? Vấn đề chính trong quá trình phát triển phần mềm.

Nhà toán học G. Polya, 1945, phương pháp giải quyết bài toán:

Bước 1. Hiểu bài toán.

Bước 2. Đặt ra kế hoạch giải quyết bài toán.

Bước 3. Thực hiện kế hoạch.

Bước 4. Đánh giá tính đúng của lời giải và khả năng dùng nó để giải các bài toán khác.

Phát hiện thuật toán

Trong ngữ cảnh phát triển phần mềm, các bước này trở thành

Bước 1. Hiểu bài toán.

Bước 2. Đưa ra ý tưởng về một thuật toán có khả năng giải quyết bài toán.

Bước 3. Trình bày rõ thuật toán và biểu diễn nó dưới dạng một chương trình.

Bước 4. Đánh giá tính đúng của chương trình và khả năng dùng nó như là công cụ để giải các bài toán khác.

Chú ý:

- Cần hạn chế quá trình thử-và-sai trong quá trình xây dựng thuật toán
- Cần có đầy đủ dữ kiện (hoàn toàn hiểu bài toán) trước khi tìm được lời giải.

Phân tích để đánh giá thuật toán

Mục tiêu của bước phân tích thuật toán:

- Kiểm tra **tính đúng đắn**: Thuật toán có giải đúng bài toán không?
- Đánh giá **tính hiệu quả**: Thuật toán có chạy nhanh và sử dụng tài nguyên hợp lý không?

Các tiêu chí đánh giá hiệu quả:

- **Thời gian**: số lượng bước tính toán cần thực hiện.
  - $\Rightarrow$ **Độ phức tạp thời gian**
- **Không gian**: lượng bộ nhớ và tài nguyên cần dùng khi thuật toán chạy.
  - $\Rightarrow$ **Độ phức tạp không gian**

Các chiến lược chứng minh tính đúng của thuật toán

1. Kiểm thử (Testing)

- Chạy thử thuật toán với các bộ dữ liệu vào cụ thể.
- **Ưu điểm:** Dễ thực hiện.
- **Nhược điểm:** Không đảm bảo phát hiện hết lỗi tiềm ẩn.

2. Chứng minh tính đúng (Correctness Proof)

- Sử dụng các lập luận toán học để chứng minh thuật toán luôn đúng.
- **Ưu điểm:** Tổng quát.
- **Nhược điểm:** Khó thực hiện và vẫn có nguy cơ sai sót.

3. Kết hợp kiểm thử và chứng minh

- Thường cho kết quả đáng tin cậy và hiệu quả hơn.

Các phương pháp chứng minh tính đúng:

- **Thuật toán đệ quy:** dùng phương pháp quy nạp.
- **Thuật toán không đệ quy:** sử dụng bất biến vòng lặp (loop invariant).

Chứng minh tính đúng đối với thuật toán đệ quy

Phương pháp quy nạp

- Chứng minh tính đúng của thuật toán dựa trên kích thước dữ liệu đầu vào.

1. Cơ sở quy nạp (Base Case)

- Xét trường hợp suy biến của đệ quy (kích thước nhỏ nhất).
- Chứng minh thuật toán trả về kết quả đúng trong trường hợp này.

2. Giả thiết quy nạp (Induction Hypothesis)

- Giả sử thuật toán đúng với dữ liệu có kích thước n.

3. Bước quy nạp (Inductive Step)

- Chứng minh rằng nếu thuật toán đúng với kích thước n, thì với kích thước $n + 1$, thuật toán cũng cho kết quả đúng.

Ví dụ: Chứng minh tính đúng của thuật toán đệ quy

Thuật toán tìm giá trị lớn nhất của dãy $A_1, A_2, \ldots, A_n$:

```
Maximum(n)
    if (n == 1)
        return A[1];
    else
        return max(Maximum(n-1), A[n]);
End.
```

Ví dụ: Chứng minh tính đúng của thuật toán đệ quy

Chứng minh tính đúng:

1. Cơ sở quy nạp:

- Khi $n = 1$, dãy chỉ có một phần tử A_1.
- Thuật toán trả lại A_1, đây là giá trị lớn nhất của dãy.

2. Giả thiết quy nạp:

- Giả sử thuật toán Maximum(n) trả lại đúng giá trị lớn nhất của $A_1, \ldots, A_n$.

3. Bước quy nạp:

- Xét dãy $A_1, \ldots, A_{n+1}$.
- Thuật toán trả lại:

$$\max(\text{Maximum}(n), A_{n+1}) = \max(A_1, A_2, \ldots, A_n, A_{n+1})$$

- Vậy thuật toán đúng với kích thước $n + 1$.

Chứng minh tính đúng đối với thuật toán không đệ quy

Phương pháp bất biến vòng lặp (Loop Invariant)

- Bất biến vòng lặp là một biểu thức logic liên quan đến các biến trong vòng lặp, và giữ nguyên giá trị đúng trong suốt quá trình lặp.
- Được sử dụng để chỉ ra thuật toán lặp dừng đúng cách và trả về kết quả chính xác.
- Với các vòng lặp lồng nhau: chứng minh từ vòng lặp trong cùng ra ngoài.

Ba đặc trưng của bất biến vòng lặp:

1. Khởi tạo

- Bất biến phải đúng trước khi vòng lặp bắt đầu (lần lặp đầu tiên).

2. Duy trì

- Nếu bất biến đúng trước một vòng lặp, nó vẫn đúng trước vòng lặp kế tiếp.

3. Kết thúc

- Khi vòng lặp kết thúc, bất biến cung cấp thông tin cần thiết để suy ra kết quả của thuật toán là đúng.

Ví dụ: Chứng minh tính đúng của thuật toán đệ quy

Thuật toán tìm giá trị lớn nhất của dãy $A_1, A_2, \ldots, A_n$:

```
Maximum(n)
        max=A[1];
        for(i=2;i<=n;i++)
            if (max< A[i])
                max = A[i];
   return max
End.
```

Ví dụ: Bất biến vòng lặp trong thuật toán tìm giá trị lớn nhất

Bất biến vòng lặp: $m_j = \max(A_1, \ldots, A_j)$

1. Khởi tạo

- Tại bước đầu tiên: $m_1 = A_1$.
- Rõ ràng: $m_1 = \max(A_1)$ - bất biến đúng.

2. Duy trì

- Giả sử tại vòng lặp thứ j:

$$m_j = \max(A_1, \ldots, A_j)$$

- Tại bước tiếp theo:

$$m_{j+1} = \max(m_j, A_{j+1}) = \max(A_1, \ldots, A_j, A_{j+1})$$

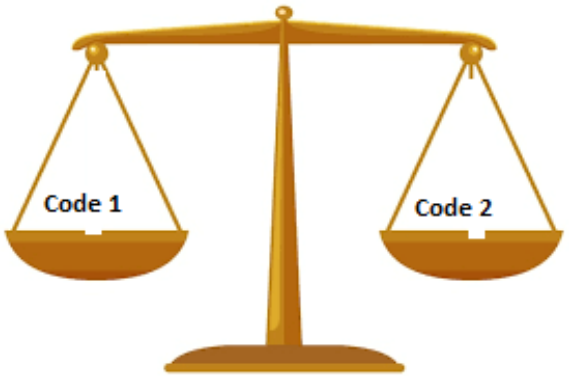
- Bất biến tiếp tục đúng.

3. Kết thúc

- Sau $t = n$ vòng lặp, chỉ số đạt $j = n$.
- Khi vòng lặp kết thúc:

$$m_n = \max(A_1, \ldots, A_n)$$

So sánh tính hiệu quả của hai thuật toán



1. **Phương pháp thực nghiệm**
   - Lập trình thuật toán, chạy thử với dữ liệu thực tế hoặc dữ liệu mô phỏng.
   - Ghi nhận các số đo về thời gian chạy và mức sử dụng tài nguyên.
   - **Ưu điểm:** Đánh giá được mọi thuật toán.
   - **Nhược điểm:** Phụ thuộc vào phần cứng (máy tính) và môi trường thực thi.
2. **Phương pháp lý thuyết**
   - Xác định quan hệ giữa thời gian thực hiện và kích thước dữ liệu thông qua hàm toán học.
   - **Ưu điểm:** Không phụ thuộc ngôn ngữ lập trình, loại máy tính và kích thước dữ liệu.
   - **Nhược điểm:** Khó vì cần mô hình hóa chi tiết và biểu diễn các ràng buộc toán học.

Đánh giá độ phức tạp thuật toán

- Đánh giá độ phức tạp của thuật toán là đánh giá lượng tài nguyên các loại mà thuật toán cần sử dụng. Đặc biệt quan tâm đến tài nguyên *thời gian tính*
- Kích thước dữ liệu đầu vào (hay độ dài dữ liệu đầu vào)
  - Số lượng phần tử trong danh sách
  - Số đỉnh, số cạnh trong một đồ thị,...

Phép toán cơ bản

Là các phép toán có thể thực hiện được với thời gian bị chặn bởi một hằng số không phụ thuộc vào kích thước dữ liệu

- Gọi n là kích thước dữ liệu đưa vào thì thời gian thực hiện thuật toán có thể biểu diễn một cách tương đối như một hàm của n: $T(n)$.
- Độ phức tạp thuật toán $T(n)$ được tính bằng cách đếm số phép toán cơ bản thực hiện thuật toán đó.

Các loại thời gian tính

- Thời gian tính tốt nhất (*best-case complexity*): số lượng phép toán cơ bản tối thiểu cần thiết để thực hiện thuật toán với mọi bộ dữ liệu đầu vào
- Thời gian tính tệ nhất (*worst-case complexity*): số lượng phép toán cơ bản nhiều nhất cần thiết để thực hiện thuật toán với mọi bộ dữ liệu đầu vào
- Thời gian tính trung bình (*average-case complexity*): số lượng phép toán cơ bản trung bình cần thiết để thực hiện thuật toán trên tập hữu hạn các đầu vào.

Các loại thời gian tính

- Cần hiểu thế nào về đánh giá thuật toán (về thời gian)
  - Là đánh giá độ phức tạp về thời gian của thật toán trong trường hợp tồi nhất
  - Ước lượng lượng thời gian lớn nhất mà thuật toán cần để đưa ra kết quả
- Dùng đánh giá này thế nào cho đúng
  - VD. Có 2 thuật toán A và B cùng giải bài toán P với độ phức tạp về thời gian trong trường hợp tồi nhất lần lượt là $T_A(n) = 100n^2 + 20$ và $T_B(n) = n^3 + 2n^2 + n$.
  - Kết luận thuật toán A luôn nhanh hơn B?
  - Trong thực tế cần chọn thuật toán nào? A hay B?
- Trong trường hợp tổng quát, cần chọn thuật toán có độ phức tạp về thời gian nhỏ hơn

Nội dung

1. Bài toán và thuật toán
2. Độ phức tạp thuật toán
3. Các ký pháp
4. Xác định độ phức tạp tính toán
5. P và NP

Kí hiệu Tiệm cận Big O, Big Theta - Θ, Big Ω

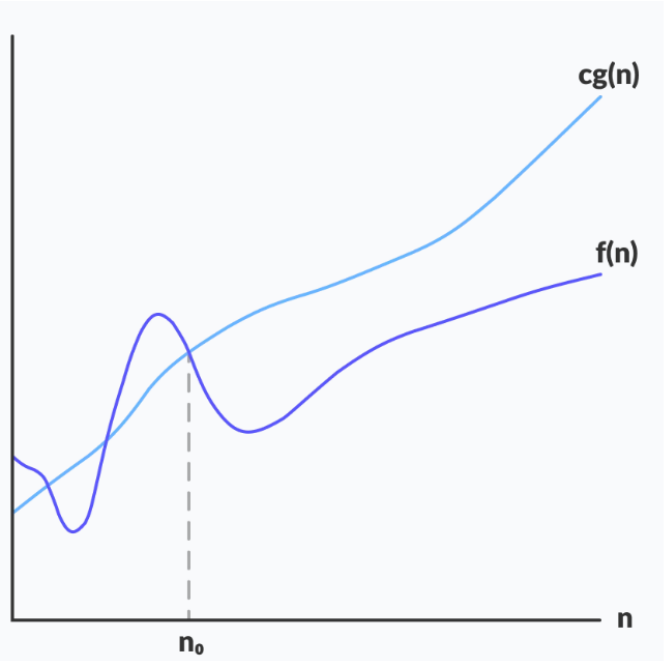
- O, Ω, Θ dùng để ký hiệu đại diện cho một tập các hàm số
- Dùng để đánh giá cận trên, cận dưới, cận chặt của thuật toán theo mức độ tăng của hàm theo kích thước đầu vào
- Mục đích là dùng để đánh giá độ phức tạp thuật toán của thuật toán thông qua các hàm đơn giản, quen thuộc.
  - Ví dụ hàm đa thức theo biến n, $nlog(n)$, n^2,...
  - Dễ dàng ước lượng, so sánh độ phức tạp của các thuật toán

Kí hiệu O lớn (Big-O) - O

- Đối với hàm $f(n)$ cho trước, ta kí hiệu $O(g(n))$ là tập các hàm

$$O(g(n)) = \{f(n) : \exists c > 0, n_0 > 0 \text{ sao cho } 0 \leq f(n) \leq c \cdot g(n)\ \forall n \geq n_0\}$$

- Ta nói rằng $g(n)$ là tiệm cận trên của $f(n)$ và viết $f(n) \in O(g(n))$ hoặc $f(n) = O(g(n))$



- **Mục tiêu là tìm hàm $g(n)$ nhỏ nhất thoả mãn $f(n) \leq c \cdot g(n)$**

Kí hiệu O lớn (Big-O) - O

Ví dụ

- $f(n) = n^2 + 1$

Kí hiệu O lớn (Big-O) - O

Ví dụ

- $f(n) = n^2 + 1$
  Ta có $n^2 + 1 \leq 2n^2, \forall n \geq 1$.
  Do đó, $n^2 + 1 = O(n^2)$ *với* $c = 2$ *và* $n_0 = 1$
- $f(n) = n^4 + 100n^2 + 50$

Kí hiệu O lớn (Big-O) - O

Ví dụ

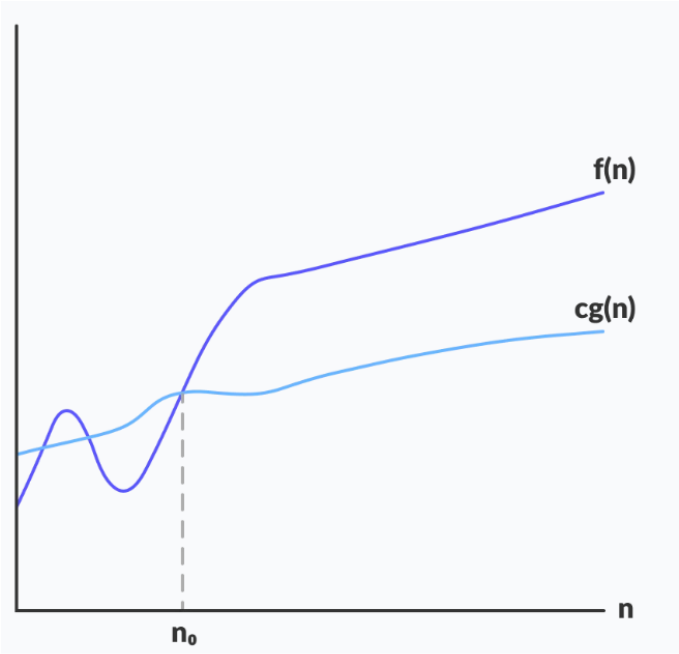
- $f(n) = n^2 + 1$
  Ta có $n^2 + 1 \leq 2n^2, \forall n \geq 1$.
  Do đó, $n^2 + 1 = O(n^2)$ *với* $c = 2$ *và* $n_0 = 1$
- $f(n) = n^4 + 100n^2 + 50$
  Ta có $n^4 + 100n^2 + 50 \leq 2n^4, \forall n \geq 11$.
  Do đó, $n^4 + 100n^2 + 50 = O(n^4)$ *với* $c = 2$ *và* $n_0 = 11$

Kí hiệu Omega lớn (Big-Ω) - Ω

- Đối với hàm $f(n)$ cho trước, ta kí hiệu $\Omega(g(n))$ là tập các hàm

$$\Omega(g(n)) = \{f(n) : \exists c > 0, n_0 > 0 \text{ sao cho } 0 \leq c \cdot g(n) \leq f(n)\ \forall n \geq n_0\}$$

- Ta nói rằng $g(n)$ là tiệm cận dưới của $f(n)$ và viết $f(n) \in \Omega(g(n))$ hoặc $f(n) = \Omega(g(n))$



- **Mục tiêu là tìm hàm $g(n)$ lớn nhất thoả mãn $c \cdot g(n) \leq f(n)$**

Kí hiệu Omega lớn (Big-Ω) - Ω

Ví dụ

- $f(n) = 5n^2$

Kí hiệu Omega lớn (Big-Ω) - Ω

Ví dụ

- $f(n) = 5n^2$
  Ta cần tìm c, n_0 *sao cho* $0 \leq cn^2 \leq 5n^2 \rightarrow c = 5, n_0 = 1$
  Do đó, $5n^2 = \Omega(n^2)$ *với* $c = 5$ *và* $n_0 = 1$
- $f(n) = 100n + 5 \neq \Omega(n^2)$

Kí hiệu Omega lớn (Big-Ω) - Ω

Ví dụ

- $f(n) = 5n^2$
  Ta cần tìm c, n_0 *sao cho* $0 \leq cn^2 \leq 5n^2 \rightarrow c = 5, n_0 = 1$
  Do đó, $5n^2 = \Omega(n^2)$ *với* $c = 5$ *và* $n_0 = 1$
- $f(n) = 100n + 5 \neq \Omega(n^2)$
  Giả sử, tồn tại c, n_0 *sao cho* $0 \leq cn^2 \leq 100n + 5$
  $100n + 5 \leq 100n + 5n(\forall n \geq 1) = 105n$
  $cn^2 \leq 105n \rightarrow n(cn - 105) \leq 0$
  Do n dương nên $cn - 105n \leq 0 \rightarrow n \leq \frac{105}{c}$
  Mâu thuẫn, n không thể nhỏ hơn một hằng số

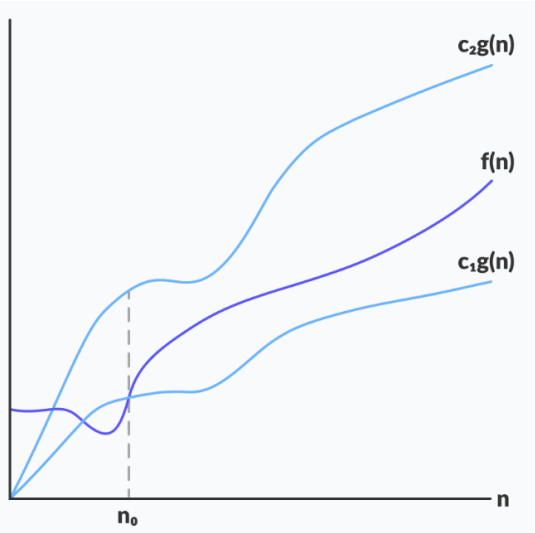
Kí hiệu Big Theta - Θ

Định nghĩa

Đối với hàm $g(n)$ cho trước, ta kí hiệu $\Theta(g(n))$ là tập các hàm

$$\Theta(g(n)) = \{f(n) : \exists c_1, c_2, n_0 \text{ sao cho } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \ \forall n \geq n_0\}$$

- Ta nói rằng $g(n)$ là đánh giá tiệm cận đúng cho $f(n)$ và viết $f(n) \in \Theta(g(n))$ hoặc $f(n) = \Theta(g(n))$



Đối với hàm đa thức: để so sánh tốc độ tăng chỉ cần nhìn vào số hạng với số mũ cao nhất

Kí hiệu Big Theta - Θ

Ví dụ

- *Chứng minh rằng* $11n^2 - 4n = \Theta(n^2)$

Kí hiệu Big Theta - Θ

Ví dụ

- *Chứng minh rằng* $11n^2 - 4n = \Theta(n^2)$
  Ta có: $n^2 \leq 11n^2 - 4n \leq 12n^2$, *với mọi* $n \geq 1$, $c_1 = 1$ *và* $c_2 = 12$
- $f(n) = \frac{n^2}{2} - \frac{n}{2}$

Kí hiệu Big Theta - Θ

Ví dụ

- *Chứng minh rằng* $11n^2 - 4n = \Theta(n^2)$
  Ta có: $n^2 \leq 11n^2 - 4n \leq 12n^2$, *với mọi* $n \geq 1$, $c_1 = 1$ *và* $c_2 = 12$
- $f(n) = \frac{n^2}{2} - \frac{n}{2}$
  Ta có: $\frac{n^2}{5} \leq \frac{n^2}{2} - \frac{n}{2} \leq n^2, \forall n \geq 2$
  Tồn tại $c_1 = \frac{1}{5}, c_2 = 1$ *và* $n_0 = 2 \rightarrow \frac{n^2}{2} - \frac{n}{2} = \Theta(n^2)$

Sử dụng kí hiệu tiệm cận Θ, Ω và O

Đối với hai hàm $g(n)$ và $f(n)$ bất kỳ,

$$f(n) = \Theta(g(n))$$

khi và chỉ khi

$$f(n) = O(g(n)) \text{và } f(n) = \Omega(g(n))$$

Cách nói về thời gian tính

- Thời gian tính là $O(g(n))$ hiểu là: Đánh giá tình huống tồi nhất là $O(g(n))$. Nghĩa là, thời gian tính trong tính huống tồi nhất được xác định bởi một hàm $f(n) \in O(g(n))$
- Thời gian tính là $\Omega((f(n))$ hiểu là: Đánh giá trong tình huống tốt nhất là $\Omega((f(n))$. Nghĩa là, thời gian tính trong tình huống tốt nhất được xác định bởi một hàm $f(n) \in \Omega(g(n))$

Các phương pháp chứng minh quan hệ tiệm cận

1. Dùng định nghĩa (c tìm được, n_0 xác định):

- Chứng minh trực tiếp bằng tìm hằng số c, n_0 sao cho bất đẳng thức đúng với mọi $n \geq n_0$.
- Thường dùng cho các hàm đơn giản (đa thức, log, hàm mũ).

2. Phương pháp quy nạp (dùng cho bất đẳng thức phụ thuộc n):

- Chứng minh cơ sở (base case) cho vài n_0.
- Giả thiết quy nạp: bất đẳng thức đúng với n.
- Bước quy nạp: suy ra bất đẳng thức đúng với $n+1$.

Ví dụ (quy nạp): $\log n = O(n)$**.** (log ở đây có thể là $\log_2$ hay ln.)

- Cơ sở: $n = 1$: $\log 1 = 0 \leq c \cdot 1$ (chọn $c \geq 1$ chẳng hạn).
- Giả thiết: $\log n \leq n$ với $n \geq 1$.
- Bước quy nạp:

$$\log(n+1) \leq \log(2n) = \log n + 1 \leq n + 1.$$

Do đó $\log(n+1) \leq n+1$. Vậy tồn tại $c = 1$ và $n_0 = 1$ sao cho $\log n \leq c\, n$ với mọi $n \geq n_0$.

Chứng minh quan hệ tiệm cận — Phương pháp giới hạn (limit)

Cho hai hàm dương $f(n)$ và $g(n)$ với $n \to \infty$. Xét giới hạn

$$L = \lim_{n\to\infty} \frac{f(n)}{g(n)}.$$

Từ giá trị L ta suy ra quan hệ tiệm cận như sau:

- Nếu $L = 0$ thì $f(n) = O(g(n))$.
- Nếu $0 < L < \infty$ thì $f(n) = \Theta(g(n))$.
- Nếu $L = \infty$ thì $f(n) = \Omega(g(n))$.

Lưu ý: phương pháp giới hạn rất tiện khi tỉ số có giới hạn hữu hạn; nếu giới hạn không tồn tại, cần dùng định nghĩa hoặc các ước lượng khác.

Nội dung

1. Bài toán và thuật toán
2. Độ phức tạp thuật toán
3. Các ký pháp
4. Xác định độ phức tạp tính toán
5. P và NP

Quy tắc bỏ hằng số

- Nếu đoạn chương trình P có thời gian thực hiện $T(n) = O(c_1 g(n))$ với c_1 là một hằng số dương thì có thể coi đoạn chương trình đó có độ phức tạp tính toán là $O(g(n))$
- Quy tắc này cũng đúng với các kí pháp Ω và Θ

Quy tắc lấy max

- Nếu đoạn chương trình P có thời gian thực hiện $T(n) = O(g_1(n) + g_2(n))$ thì có thể coi đoạn chương trình có độ phức tạp tính toán bằng $O(max(g_1(n), g_2(n)))$
- Quy tắc này cũng đúng với các kí pháp Ω và Θ

Quy tắc cộng

- Nếu đoạn chương trình P_1 có thời gian thực hiện $T_1(n) = O(g_1(n))$, đoạn chương trình P_2 có thời gian thực hiện là $T_2(n) = O(g_2(n))$ thì thời gian thực hiện đoạn chương trình P_1 rồi đến P_2 sẽ là

$$T_1(n) + T_2(n) = O(g_1(n) + g_2(n))$$

- Quy tắc này cũng đúng với các kí pháp Ω và Θ

Quy tắc nhân

- Nếu đoạn chương trình P có thời gian thực hiện $T(n) = O(g_1(n))$. Khi đó, nếu thực hiện $k(n)$ lần đoạn chương trình P với $k(n) = O(g_2(n))$ thì độ phức tạp tính toán sẽ là $O(g_1(n)g_2(n))$
- Quy tắc này cũng đúng với các kí pháp Ω và Θ

Đánh giá độ phức tạp của chương trình

Một số công thức tổng chuỗi

- $S_n = 1 + 2 + 3 + ... + n = \frac{n \times (n+1)}{2}$
- $S_n = 1^2 + 2^2 + 3^2 + ... + n^2 = \frac{n \times (n+1)(2n+1)}{6}$
- $S_n = 1^3 + 2^3 + 3^3 + ... + n^3 = \frac{n^2 \times (n+1)^2}{4}$
- $S_n = 1^4 + 2^4 + 3^4 + ... + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$

Đánh giá độ phức tạp của chương trình

```
for (i = 0; i < n; i++)
    for (j = 0; j < n; j++)
        count++;
```

Đánh giá độ phức tạp của chương trình

```
// Lặp n lần
for (i = 0; i < n; i++)
    // Lặp n lần
    for (j = 0; j < n; j++)
        count++;
```

Đánh giá độ phức tạp của chương trình

```
for (i = n/2; i <=n; i++)
    for (j = 1; j + n/2 <=n; j++)
        for (k = 1; k <=n; k = k*2)
            count++;
```

Đánh giá độ phức tạp của chương trình

```
// Lặp n/2 lần
for (i = n/2; i <=n; i++)
    // Lặp n/2 lần
    for (j = 1; j + n/2 <=n; j++)
        // Lặp logn lần
        for (k = 1; k <=n; k = k*2)
            count++;
```

Đánh giá độ phức tạp của chương trình

```
x = x + 1
for (i = 1; i <= n; i++)
    m = m + 2

for (j = 1; j <=n; j++)
        for (k = 1; k<=n; k++)
            count++;
```

Đánh giá độ phức tạp của chương trình

```
x = x + 1 // Hằng số
// Lặp n lần
for (i = 1; i <= n; i++)
    m = m + 2; // Hằng số

// Lặp n lần
for (j = 1; j <=n; j++)
        for (k = 1; k<=n; k++) // Lặp n lần
            count++; // Hằng số
```

Đánh giá độ phức tạp của chương trình

```
for (i = n/2; i <=n; i++)
    for (j = 1; j <= n; j = j*2)
        for (k = 1; k <=n; k = k*2)
            count++;
```

Đánh giá độ phức tạp của chương trình

```
//Lặp n/2 lần
for (i = n/2; i <=n; i++)
    //Lặp logn lần
    for (j = 1; j <= n; j = j*2)
        // Lặp logn lần
        for (k = 1; k <=n; k = k*2)
            count++;
```

Một số tính chất của hàm mũ, loga và giai thừa

- Hàm mũ

$$a = b^{log_b a}, log_b a = 1/log_a b$$

- Trong kí hiệu tiệm cận cơ số của log là không quan trọng

$$O(lgn) = O(lnn) = O(logn)$$

- Công thức Stirling

$$n! = \sqrt{2\pi n}(\frac{n}{e})^n(1 + \Theta(\frac{1}{n}))$$

- Giai thừa và hàm mũ:

$$2^n < n! < n^n \text{ với } n > 5$$

$$logn! = \Theta(nlogn)$$

Lớp hàm đánh giá

Hàm 1 - Độ phức tạp hằng số $\mathcal{O}(1)$

- Số phép tính, thời gian chạy hoặc dung lượng bộ nhớ không phụ thuộc vào độ lớn đầu vào.
- Thuật toán với số hữu hạn thao tác được thực hiện một hoặc một vài lần.
- Ví dụ: thuật toán giải phương trình bậc nhất, phương trình bậc hai.

Hàm logn - Độ phức tạp logarit $\mathcal{O}(\log n)$

- Thời gian thực hiện tăng theo kích thước dữ liệu vào với tốc độ hàm logarit.
- Ví dụ: thuật toán tìm kiếm nhị phân trên mảng đã sắp xếp; các thao tác trên nhánh con của cây nhị phân đầy đủ.

Lớp hàm đánh giá

Hàm n - Độ phức tạp tuyến tính $\mathcal{O}(n)$

- Thời gian thực hiện tăng theo kích thước dữ liệu đầu vào với tốc độ tuyến tính.
- Thường gồm một số hữu hạn thao tác lặp qua toàn bộ dữ liệu.
- Ví dụ: tìm kiếm tuần tự (tìm phần tử, tìm giá trị lớn nhất/nhỏ nhất) trên mảng.

Hàm nlogn - Độ phức tạp tuyến tính–logarit $\mathcal{O}(n \log n)$

- Các thuật toán chia bài toán thành các phần nhỏ hơn, giải độc lập, rồi hợp lại (divide-and-conquer).
- Ví dụ: thuật toán sắp xếp nhanh (QuickSort), sắp xếp vun đống (HeapSort).

Các lớp hàm đánh giá

Hàm n^m - Độ phức tạp đa thức $\mathcal{O}(n^m)$

- Thuật toán có các thao tác thực hiện trong những vòng lặp lồng nhau.
- Trường hợp tổng quát: $\mathcal{O}(n^m)$ với m là số mức lặp; thường chỉ xét đến $m = 2, 3, 4$.
- Ví dụ: thuật toán sắp xếp nổi bọt (Bubble Sort), phép nhân ma trận.

Hàm 2^n - Độ phức tạp lũy thừa $\mathcal{O}(2^n)$

- Thuộc lớp thuật toán có độ phức tạp rất lớn.
- Thường xuất hiện trong các thuật toán đệ quy, duyệt toàn bộ tập con, hoặc bài toán có không gian lời giải tăng theo cấp số nhân.
- Khi n đủ lớn, bài toán gần như không thể giải được trong thời gian hữu hạn.

Định lý chính/Định lý thợ

Định lý chính/Định lý thợ (*master theorem*)

- Dùng để giải các công thức đệ quy dạng $T(n) = aT(\frac{n}{b}) + \Theta(n^k log^p n)$, với $a \geq 1, b > 1, k \geq 0$
- Bài toán ban đầu được chia thành a bài toán con có kích thước mỗi bài toán con là $\frac{n}{b}$ chi phí để tổng hợp các bài toán con là $\Theta(n^k log^p n)$
  - Nếu $a > b^k$, $T(n) = \Theta(n^{log_b a})$
  - Nếu $a = b^k$
    - Nếu $p > -1$, $T(n) = \Theta(n^{log_b a} log^{p+1} n)$
    - Nếu $p = -1$, $T(n) = \Theta(n^{log_b a} loglogn)$
    - Nếu $p < -1$, $T(n) = \Theta(n^{log_b a})$
  - Nếu $a < b^k$
    - Nếu $p \geq 0$, $T(n) = \Theta(n^k log^p n)$
    - Nếu $p < 0$, $T(n) = O(n^k)$

P và NP

1. Bài toán tối ưu

- Mục tiêu: tìm lời giải sao cho một hàm mục tiêu đạt giá trị tốt nhất (tối đa hoặc tối thiểu).
- Ví dụ: Bài toán người du lịch (TSP), Bài toán xếp ba lô, Tô màu đồ thị.

2. Bài toán quyết định

- Bài toán có hai khả năng trả lời: **YES** hoặc **NO** (hoặc True/False, 0/1).
- Nhiều bài toán có thể biểu diễn dưới cả hai dạng tối ưu và quyết định.

Chuyển đổi giữa tối ưu và quyết định

- Với bài toán tối ưu, ta có thể tạo ra một phát biểu quyết định bằng cách thêm ngưỡng k.
- **Ví dụ:**
  - TSP tối ưu $\rightarrow$ Bài toán quyết định:

"Có đường đi nào có tổng chi phí $\leq k$ hay không?"

P và NP

Định nghĩa bài toán P

Một bài toán quyết định được gọi là thuộc lớp P nếu tồn tại một thuật toán giải bài toán trong thời gian $O(n^c)$ với c là một hằng số không phụ thuộc vào kích thước đầu vào.

Ta sẽ dùng $poly(n)$ để thay thế cho $O(n^c)$

- Một đặc điểm chung của nhiều bài toán quyết định khó giải là nếu ai đó cho chúng ta lời giải của bài toán đó, việc kiểm tra xem đó có phải là lời giải đúng hay không sẽ **dễ hơn** việc trực tiếp đi giải bài toán
- Ví dụ bài toán independent set: Nếu ai cho ta một tập đỉnh X của đồ thị và nói rằng tập đỉnh đó là tập độc lập của đồ thị có kích thước ít nhất là k, ta có thể kiểm tra trong *poly(n)* xem X có phải tập độc lập hay không và $|X| \leq k$ hay không?

P và NP

- Tập X trên gọi là một **bằng chứng** cho thấy input của bài toán có output là Yes. Hai tính chất quan trọng của X mà ta có thể rút ra
  - Ngắn gọn: Ta mô tả X sử dụng tối đa *poly(n)* vì $|X| \leq n$
  - Dễ kiểm tra: Ta có thể kiểm tra X có thoả mãn điều kiện là một tập độc lập có kích thước là k hay không trong thời gian *poly(n)*
- Ta gọi X là một bằng chứng ngắn gọn dễ kiểm tra của bài toán tập độc lập

P và NP

Bài toán NP

Một bài toán quyết định được gọi là thuộc lớp NP nếu tồn tại một bằng chứng ngắn gọn, dễ kiểm tra cho câu trả lời Yes của bài toán đó

Theo định nghĩa trên, để chứng minh một bài toán thuộc lớp NP ta cần:

- Chỉ ra bằng chứng ngắn gọn cho đầu vào Yes
  - Chỉ ra một thuật toán kiểm tra bằng chứng đó trong thời gian *poly(n)*

P và NP

- Về mặt trực quan, một bài toán dễ giải thì cũng dễ kiểm tra lời giải, do đó $P \subset NP$,
- Câu hỏi thách thức P = NP